

FHEStore: Bridging Persistent Data and FHE Accelerators with FPGA-Based Computational Storage

Anonymous Author(s)

Abstract

Fully homomorphic encryption (FHE) accelerators act as ciphertext consumers, substantially reducing the cost of homomorphic computation. However, producing their inputs from persistent data and processing returned results introduce additional overhead. Conventionally, the host serves as the input producer, using CPU and memory resources for data access, preprocessing, encryption, and buffering. It also decrypts returned results and writes them back to storage. Moreover, ciphertext expansion further increases buffering and data-movement costs.

We present FHEStore, an FPGA-based computational storage system that bridges persistent data and FHE accelerators through storage-side input production and result postprocessing. FHEStore combines storage-side data access with an FPGA streaming datapath to integrate lightweight preprocessing and Fast Fully Homomorphic Encryption over the Torus (TFHE). On the return path, it decrypts results on the FPGA and writes the decrypted results back to persistent storage.

We prototype FHEStore on an FPGA-enabled computational storage device and integrate it with a remote homomorphic processing unit (HPU). For TFHE encryption of unsigned 8-bit data with a batch size of 4 KiB, the FPGA encryption engine achieves a $5.33\times$ speedup over host CPU-based encryption. In the evaluated end-to-end input production workflow, FHEStore achieves a $3.26\times$ speedup over the host-based baseline. We also demonstrate homomorphic computation on the remote HPU using FHEStore-generated ciphertexts.

CCS Concepts

• **Hardware** → **Reconfigurable logic and FPGAs**; • **Information systems** → *Storage architectures*; • **Security and privacy** → *Cryptography*.

Keywords

Fully homomorphic encryption, TFHE, FPGA, high-level synthesis, computational storage, streaming dataflow

ACM Reference Format:

Anonymous Author(s). 2027. FHEStore: Bridging Persistent Data and FHE Accelerators with FPGA-Based Computational Storage. In *Proceedings of 35th ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '27)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

Fully homomorphic encryption (FHE) enables a data owner to outsource computation over encrypted data without revealing the plaintext to the evaluator. For applications backed by persistent data, using an FHE accelerator involves two paths around homomorphic computation. An input producer accesses application data

and generates the ciphertexts that the accelerator consumes. On the return path, result ciphertexts are decrypted and the recovered values are written back to storage. Input production and result postprocessing thus connect persistent application data with encrypted computation, and their costs remain part of the application workflow.

Dedicated hardware research addresses the execution cost of plaintext-ciphertext conversion through specialized encryption and decryption architectures [2, 8, 9, 12]. Specialized arithmetic and parallel datapaths make cryptographic conversion a practical acceleration target across FHE schemes. Translating that acceleration into an application benefit also requires connecting the operators to data preparation, ciphertext delivery, and result recovery. The cost of those surrounding stages depends on where conversion is placed and how data are handed between stages.

Research on memory-aware FHE accelerators complements arithmetic acceleration by optimizing data reuse, memory access, and traffic during homomorphic computation [5, 6, 13, 16, 18]. This work establishes the expanded ciphertext representation and large cryptographic working sets, including evaluation keys, as architectural concerns alongside arithmetic. The resulting data layouts, memory organizations, and streaming schedules improve the execution of homomorphic programs. Their optimization boundary centers on the working set consumed by encrypted computation. Application workflows additionally need to generate input ciphertexts from persistent data and convert returned ciphertexts into persistent results; organizing these paths is a complementary systems problem.

Client-side systems further combine cryptographic acceleration with encoding, memory organization, streaming, buffering, or client-server communication [4, 7, 14, 17]. Their client-side processing paths connect cryptographic stages to memory or communication interfaces. For applications whose inputs and outputs reside in persistent storage, the design boundary also encompasses data access and preprocessing before encryption, as well as result write-back after decryption. Integrating these stages with conversion extends the coordination problem from client processing to persistent-data input production and result postprocessing.

This broader boundary couples cryptographic execution with host resource use. In a conventional host-based workflow, CPU software reads and preprocesses stored plaintext, encodes and encrypts the values, and stages ciphertexts for accelerator delivery. It also decrypts returned ciphertexts and writes the recovered values back to storage. Host memory holds plaintext, ciphertexts, and intermediate buffers across these stages; ciphertext expansion increases the volume handled at their interfaces. Offloading encryption and decryption moves their cryptographic computation off the host CPU, while the placement and coordination of the operators determine the buffering and data handoffs required around them. Our objective is therefore to accelerate TFHE encryption and decryption while reducing host CPU work, host-memory buffering,

and intermediate data movement between persistent storage and FHE accelerators. Meeting this objective requires jointly designing cryptographic operators and their placement within the input and return paths.

We present FHEStore, an FPGA-based computational storage system that integrates cryptographic acceleration with storage-side input production and result postprocessing. FHEStore implements TFHE encryption and decryption [1] as hardware operators using high-level synthesis (HLS), with plaintext encoding integrated into encryption. The input streaming datapath connects storage-side data access and lightweight preprocessing to FPGA ciphertext generation. The return datapath connects FPGA decryption of returned ciphertexts to persistent result write-back. A control path coordinates operator execution and data handling, supporting an application workflow from persistent inputs through ciphertext handoff to recovered-result write-back.

We summarize the contributions of this work as follows:

- **HLS encryption and decryption.** We develop streaming TFHE operators that integrate plaintext encoding with encryption and offload encryption and decryption from the host CPU.
- **Streaming datapaths and application control.** We organize storage access, lightweight preprocessing, and cryptographic operators into storage-side input and return streams, with control supporting ciphertext handoff and decrypted-result write-back for applications.
- **Prototype and evaluation.** We prototype FHEStore on an FPGA-enabled computational storage device and demonstrate computation on a remote homomorphic processing unit (HPU) using FHEStore-generated ciphertexts. For unsigned 8-bit inputs in 4 KiB plaintext batches, FPGA TFHE encryption and end-to-end input production achieve 5.33× and 3.26× speedups over host CPU encryption and host input production, respectively.

2 Background and Motivation

2.1 FHE and TFHE Fundamentals

FHE provides key generation, encryption, homomorphic evaluation, and decryption [3]. Let k_{enc} , k_{dec} , and k_{eval} denote the scheme's encryption, decryption, and evaluation key material. For messages $m_1, \dots, m_t$, form $c_i \leftarrow \text{Enc}_{k_{\text{enc}}}(m_i)$ and $c_f \leftarrow \text{Eval}_{k_{\text{eval}}}(f, c_1, \dots, c_t)$. With valid parameters, an exact scheme satisfies the following relation for a supported computation f , with overwhelming probability over its randomness:

$$\text{Dec}_{k_{\text{dec}}}(c_f) = f(m_1, \dots, m_t). \quad (1)$$

Encoding maps application values into the scheme's message representation; decoding maps decrypted messages back into application values.

TFHE operates over the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ and uses learning-with-errors (LWE) and ring-based ciphertexts, with bootstrapping to control noise during homomorphic computation [1]. A representative building block is secret-key torus-LWE encryption. Let $\mu = \text{Encode}(m)$ belong to a discrete message space $\mathcal{M} \subset \mathbb{T}$, and let $s \in \mathbb{Z}^n$ be the secret key. Sampling a uniform mask $a \in \mathbb{T}^n$ and an error e from the scheme's noise distribution yields the ciphertext

$$c = (a, b), \quad b = \langle a, s \rangle + \mu + e \pmod{1}. \quad (2)$$

Decryption computes the ciphertext's phase,

$$\phi_s(c) = b - \langle a, s \rangle = \mu + e \pmod{1}, \quad (3)$$

and recovers the nearest element of $\mathcal{M}$ under the torus distance before decoding the application value. Recovery succeeds when the phase remains within the encoded message's decision region [1].

These operations expose distinct tasks for hardware: encoding, randomness generation, and key-dependent accumulation on encryption; phase recovery and decoding on decryption. In this torus-LWE representation, a ciphertext contains $n + 1$ coefficients. Its serialized size depends on the coefficient precision and cryptographic parameters, while the number of ciphertexts needed for application data depends on the encoding. These choices determine both the cryptographic workload and the volume handled by buffers and transfer interfaces.

2.2 Existing FHE Producer–Consumer Workflows

Outsourced FHE systems commonly place encoding, encryption, decryption, and decoding in the data owner's trusted domain, while a server performs homomorphic computation [4, 7, 17]. We describe this organization through a producer–consumer view: the input producer prepares ciphertexts, the evaluator consumes them to execute an encrypted program, and the owner recovers the returned results. The evaluator receives the ciphertexts and any required evaluation material, while the decryption key remains within the owner's trusted domain.

Existing implementations realize these roles through different processing and memory interfaces. Client-side accelerators connect encoding and cryptographic conversion to software control, local memory, or external-memory transfers [7, 17]. FHE computation accelerators schedule encrypted operations over ciphertexts and evaluation keys supplied to their memory systems [13, 16, 18]. Across this boundary, the producer and consumer must agree on cryptographic parameters, message encoding, and ciphertext layout. Client-aided execution can also introduce intermediate exchanges in which the client decrypts, processes, and re-encrypts values before computation resumes [14].

For the persistent-data workload considered here, input production begins with stored application data and includes access and preprocessing before encoding and encryption. Result postprocessing recovers application values through decryption and decoding and writes them back to persistent storage. Storage access, cryptographic conversion, and accelerator communication consequently form connected stages. Their interfaces determine where plaintext and ciphertext batches are buffered, how work is submitted, and when results become available to the application. This workflow provides the system context for placing conversion alongside persistent-data access.

2.3 Computational Storage and Application Integration

Computational storage couples processing with storage to offload host work or reduce data movement [11]. By executing operations along the storage-access path, it can process data where they are stored or accessed, instead of requiring host software to perform

every transformation after a conventional read. The processing interface connects persistent data to an executable task, with task completion coordinated with the surrounding application.

Typical operations include filtering, statistical aggregation, feature selection, and decompression [10]. These operations combine storage access with application processing: a read can deliver selected or transformed data, and a write can incorporate processing before data are stored. The benefit depends on the operation and its output representation. Filtering can shrink the returned data, whereas a representation-changing operation can produce a larger output.

Application integration requires abstractions for accessing data, configuring processing, and obtaining results. FPGA storage frameworks provide examples through virtual-file access, C++ interfaces with data and parameter streams, and unified HLS interfaces for memory and storage [10, 15]. Such interfaces allow application code to express processing tasks and data access while the framework coordinates transfers and hardware execution. Usability therefore depends on how the system exposes task parameters, input and output locations, and completion semantics alongside the processing operators.

This combination of data access and programmable processing suits the FHE workflow described above. On the input path, storage access and preprocessing can feed encoding and encryption; on the return path, decryption and decoding can feed result write-back. Encryption expands the representation, so operator placement determines the intermediate handoffs and buffering around the required ciphertext exchange. Integrating cryptographic operators with storage-side streams and application control is the basis of FHEStore’s architecture, presented in Section 3.

3 System Overview

FHEStore connects persistent-data access to storage-side input production and result postprocessing. Its input path connects storage access and optional preprocessing to FPGA encoding and TFHE encryption; its return path connects FPGA decryption and decoding to result write-back. The host coordinates CSD execution and ciphertext handoffs to and from the external consumer.

3.1 System Organization

Figure 1 shows the system boundaries and the control and data paths. FHEStore comprises an FPGA-based computational storage device (CSD) and the host software that coordinates its operators and ciphertext handoffs. The remote server represents an external prototype for privacy-preserving outsourced computation with a homomorphic accelerator. The CSD combines persistent storage, device-local buffers, a device runtime, and an FPGA operator pool. The input producer reads stored data, applies lightweight preprocessing when requested, and supplies application values to encoding and encryption. On the return path, decryption and decoding recover application values for persistent write-back.

The host coordinates application control and ciphertext communication. Its task runtime submits device work and receives execution completion. Its ciphertext relay retrieves generated ciphertexts from the CSD and stages returned ciphertexts for the CSD’s decryption path. PCIe carries task submission, completion,

and ciphertext transfers between the host and CSD; the host connects the relay to the external service over a network.

Following the producer–consumer organization in Section 2, FHEStore assumes that the host and CSD belong to the data owner’s trusted domain. Input plaintext, recovered results, and secret-key material are handled within this domain. The host loads the LWE secret key s locally and supplies it to the CSD’s cryptographic operators through their execution contexts.

3.2 Application Task Model

An application specifies a task $T = (D, Q, E, O)$. D identifies the stored input and its record schema. Q specifies selection predicates and projected fields when preprocessing is requested. E identifies the encoding and cryptographic parameter preset. O specifies ciphertext delivery or an outsourced operation and a destination for recovered results. These descriptions express the application request independently of device endpoints, batch sizes, and transfer settings chosen by the runtime.

For example, a selective-update task can read a range of records, select those satisfying predicates over integer fields, and encrypt a projected unsigned 8-bit field. The task also specifies where the recovered values should be written after the outsourced operation.

For a selective task, let $r_0, \dots, r_{N-1}$ denote the source records. The predicate in Q selects M records, and π_Q projects their chosen field. Let $I_Q = (i_0, \dots, i_{M-1})$ list their indices in source order. With configuration E and local secret key s , the input producer emits

$$c_j \leftarrow \text{Enc}_{s,E} \left(\text{Encode}_E \left(\pi_Q(r_{i_j}) \right) \right), \quad 0 \leq j < M. \quad (4)$$

Here, c_j denotes the ciphertext bundle representing one application value. Encoding produces the message blocks for that value, and encryption applies to each block using the LWE representation in Equation 2. The host retains I_Q alongside the ciphertext sequence. For an elementwise update, the j th returned value corresponds to record i_j , allowing result postprocessing to place it in the selected field of that record.

3.3 Operator Graphs and Device Execution

The task runtime translates T into an operator program, operator contexts, and input and output memory ranges. The program specifies operator types and their stream connections. Contexts carry the record layout from D , selection and projection settings from Q , and cryptographic configuration from E . Memory ranges bind an invocation to device-local buffers and byte extents. The host submits these objects to the device runtime, which configures FPGA execution.

For selective input production, the graph connects filtering and projection directly to encoding and encryption, keeping their data handoff inside the CSD. The selected count and the per-value ciphertext size specified by E determine the generation program’s output extent. The selected record indices provide the mapping I_Q used for result placement. Tasks without preprocessing use a single cryptographic operator.

The output action O determines how execution proceeds after ciphertext generation. For tasks that invoke outsourced computation, the host relays generated ciphertexts to the external service and transfers returned ciphertexts into the input range of a separate

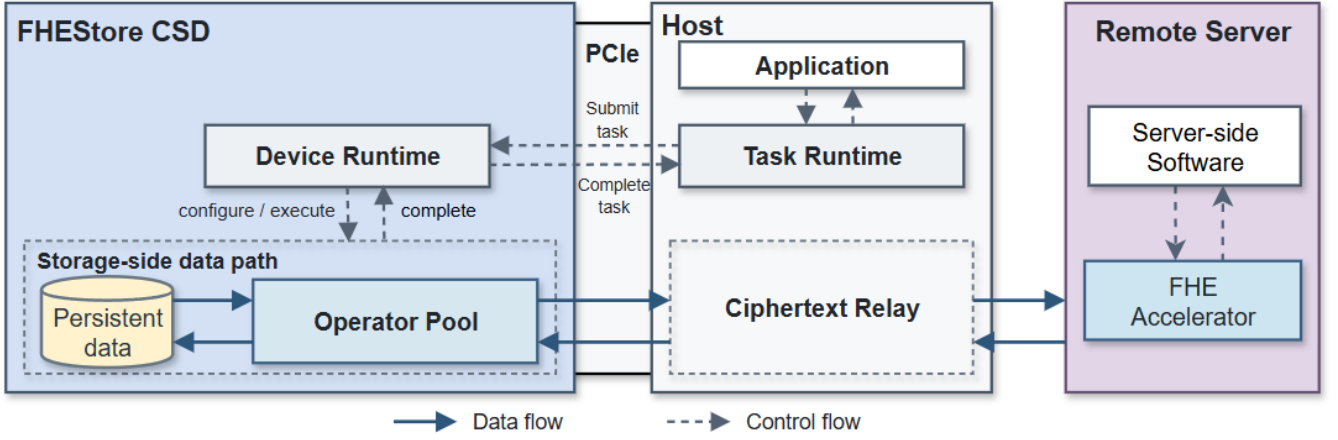


Figure 1: FHESore system organization. FHESore comprises the CSD and coordinating host software, connected over PCIe. The host relays ciphertexts over a network to an external computation service and returns result ciphertexts to the CSD. Blue arrows denote data flow; gray arrows denote control flow, including task submission and completion.

CSD decryption program. FPGA decryption and decoding produce recovered application values in its output range. For contiguous outputs, the device copies this buffer to the designated SSD range. For selected-record updates, the host uses I_Q to merge recovered values into destination record pages while preserving unselected records and other fields.

Completion is reported to the application when the output requested by O is available. A generation-only task completes with its ciphertexts and, when selection is used, its record mapping; an update task also waits for result recovery and destination I/O.

4 FPGA Design and Implementation

4.1 Integrated Encoding and Encryption

The FPGA encryption engine transforms preprocessed application values into TFHE ciphertexts, with encoding integrated into the encryption implementation. The engine is implemented using high-level synthesis (HLS). Its measured encryption performance is reported separately from the storage-side input-preparation path so that hardware computation and system integration can be assessed at their respective boundaries.

Writing guide: Present the HLS datapath and the implemented TFHE encryption algorithm. Specify arithmetic precision, encoding, key and parameter storage, randomness generation, and hardware parallelism. Document only optimizations present in the design and verify that randomness meets the scheme’s requirements.

4.2 Result Decryption Engine

The FPGA decryption engine recovers plaintext results from ciphertexts returned by the remote HPU. Its output feeds persistent write-back, completing the storage-side result-processing path.

Writing guide: Describe the HLS decryption datapath, supported result representation, decoding, key access, and handoff to storage write-back. Report its latency, throughput, and resource usage separately when measurements are available.

Table 1: Implementation details to complete before submission.

Item	Status
FPGA board and device	To be supplied
Storage-side execution platform	To be supplied
HLS tool and version	To be supplied
Achieved clock frequency	To be supplied
LUT / FF / BRAM / DSP use	To be supplied
TFHE parameters	To be supplied
Host CPU and software	To be supplied
Storage and interconnect	To be supplied
Remote HPU and transport	To be supplied
Result write-back semantics	To be supplied

4.3 HLS Mapping and Hardware Integration

The prototype places the FPGA engines on an FPGA-enabled computational storage device and connects the system to a remote HPU. Table 1 lists the implementation details needed to interpret and reproduce the reported measurements.

Writing guide: Report the HLS tool version, requested and achieved clock frequencies, initiation intervals, buffers, and interfaces for each engine. Separate synthesis estimates from measured implementation results. Identify resources shared by the engines.

5 Experimental Evaluation

5.1 Experimental Setup and Baselines

The evaluation examines FPGA encryption performance, end-to-end input preparation, and interoperability with the remote HPU. The engine comparison measures the efficiency of FPGA TFHE encryption. The input-preparation comparison measures FHESore’s storage-side input path, and the HPU demonstration exercises homomorphic computation with its generated ciphertexts.

Table 2: TFHE encryption and end-to-end input-preparation speedups over the corresponding host baselines for unsigned 8-bit inputs with a 4 KiB plaintext batch.

Measurement	Batch	Speedup
FPGA encryption vs. host CPU encryption	4 KiB	5.33×
End-to-end input preparation vs. host baseline	4 KiB	3.26×

Writing guide: Complete Table 1. Specify the CPU baseline library, thread count, compiler settings, equivalent TFHE parameters and work, batch interpretation, cache state, repetitions, and summary statistic. Describe the storage input, preprocessing workload, HPU operation, and output destination.

5.2 Metrics and Measurement Boundaries

Latency speedup is defined as

$$S_b = \frac{T_{\text{host},b}}{T_{\text{FHEStore},b}}, \quad (5)$$

where b identifies the measurement boundary and both latencies cover the same operation at that boundary. The encryption-engine comparison measures FPGA encryption against host CPU-based encryption. The end-to-end input-preparation comparison measures FHEStore’s input path against the corresponding host-based path. Remote HPU computation, returned-result processing, and persistent result write-back are separate evaluation boundaries.

Writing guide: State exact start and stop points and report absolute latencies for both measured speedups. Clarify the inclusion of storage I/O, preprocessing, setup, transfers, and ciphertext delivery. For a complete workflow measurement, time from input access to the specified result-persistence point; do not sum stage times if execution overlaps.

5.3 TFHE Encryption and Input-Preparation Results

Table 2 reports encryption and input-preparation speedups for u8 data with a 4 KiB (4096-byte) plaintext-input batch.

The FPGA encryption engine achieves a 5.33× speedup over host CPU-based encryption. At the system level, FHEStore achieves a 3.26× speedup in end-to-end input preparation over the host-based baseline. These results establish faster encryption and faster preparation of persistent inputs in the evaluated configuration.

Writing guide: Add absolute latency and throughput results, repetitions or uncertainty estimates, and a stage breakdown. Specify whether throughput is expressed as plaintext bytes, application values, or ciphertext bytes per second.

5.4 Remote HPU Demonstration

We demonstrate homomorphic computation on the remote HPU using ciphertexts generated by FHEStore. The demonstration connects storage-side input preparation to the HPU interface.

Writing guide: Describe the demonstrated HPU operation, parameters, input values, and execution procedure. Compare decrypted outputs with plaintext reference results, and state whether the demonstration

includes FPGA result decryption and persistent write-back. Report transfer and HPU execution times separately if measured.

5.5 Result Decryption and Write-Back

FHEStore’s output path receives result ciphertexts, decrypts them on the FPGA, and writes the plaintext results to persistent storage. The corresponding latency boundary runs from result receipt through decryption to the specified storage-completion point.

Writing guide: Measure FPGA decryption, result handling, storage write-back, and combined output-postprocessing latency against the corresponding host-based path. Verify persisted output values and state the persistence completion condition. Do not claim an output-path or full-workflow speedup without these measurements.

5.6 Host CPU Utilization and Data Movement

Writing guide: Measure host CPU utilization consistently across the baseline and FHEStore for input preparation and output postprocessing. Report absolute utilization and define any reduction. Measure plaintext and ciphertext traffic at each relevant boundary, including the remote HPU connection. No utilization or traffic-reduction percentage is currently available.

5.7 Placement, Selectivity, and Batch Size

Writing guide: Compare supported alternative placements, vary preprocessing selectivity where applicable, and sweep batch sizes. Measure latency, throughput, host utilization, and transfer volume. Separate FPGA acceleration from storage-side placement with an ablation where supported, and examine result size and remote transport effects.

5.8 Correctness and Limitations

The performance measurements use unsigned 8-bit inputs with a 4 KiB plaintext batch and characterize TFHE encryption and input preparation. The remote HPU demonstration establishes integration for the demonstrated computation. Output-postprocessing latency and complete read–compute–write latency remain to be measured.

Writing guide: Describe encryption/decryption checks, HPU compatibility tests, and verification of persisted results. State workload and parameter coverage, trust assumptions, and conditions under which storage access, ciphertext expansion, transport, or result write-back limits performance.

6 Related Work

FHEStore connects persistent data to FHE accelerators through storage-side input preparation and result postprocessing. The relevant comparison axes are computation scope, data-path placement, and accelerator integration. These axes connect FHEStore to work on FHE computation accelerators, FPGA encryption/decryption engines, and computational storage.

Writing guide: Add primary-paper citations and compare input/output scope, storage-side placement, remote accelerator integration, cryptographic parameters, and measured operations. Distinguish input-encryption acceleration from homomorphic-computation acceleration, and compare speedups only at compatible measurement boundaries.

7 Conclusion

FHEStore bridges persistent data and FHE accelerators through FPGA-based computational storage. Its input path combines storage-side data access, lightweight preprocessing, and TFHE encryption; its output path decrypts returned result ciphertexts on the FPGA and writes the decrypted results to persistent storage. The prototype is integrated with a remote HPU and demonstrates homomorphic computation using FHEStore-generated ciphertexts. For unsigned 8-bit data with a 4 KiB batch, the FPGA encryption engine achieves a $5.33\times$ speedup over host CPU-based encryption, and the evaluated end-to-end input preparation achieves a $3.26\times$ speedup over the host-based baseline.

References

- [1] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology* 33 (2020), 34–91. doi:10.1007/s00145-019-09319-x
- [2] Najmi Azzahra Feryputri, Infall Syafalni, Nana Sutisna, Nur Ahmadi, Rahmat Mulyawan, and Trio Adiono. 2025. FPGA-Based Hardware Accelerator for LWE Encryption and Decryption with TFHE Scheme. In *2025 IEEE 38th International System-on-Chip Conference (SOCC)*. IEEE, 1–6. doi:10.1109/SOCC66126.2025.11235379
- [3] Craig Gentry. 2009. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing (STOC '09)*. Association for Computing Machinery, New York, NY, USA, 169–178. doi:10.1145/1536414.1536440
- [4] Saransh Gupta, Rosario Cammarota, and Tajana Šimunić. 2024. MemFHE: End-to-end Computing with Fully Homomorphic Encryption in Memory. *ACM Transactions on Embedded Computing Systems* 23, 2, Article 28 (2024), 23 pages. doi:10.1145/3569955
- [5] Jongmin Kim, Gwangho Lee, Sangpyo Kim, Gina Sohn, Minsoo Rhu, John Kim, and Jung Ho Ahn. 2022. ARK: Fully Homomorphic Encryption Accelerator with Runtime Data Generation and Inter-Operation Key Reuse. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1237–1254. doi:10.1109/MICRO56248.2022.00086
- [6] Sangpyo Kim, Jongmin Kim, Michael Jaemin Kim, Wonkyung Jung, John Kim, Minsoo Rhu, and Jung Ho Ahn. 2022. BTS: An Accelerator for Bootstrappable Fully Homomorphic Encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 711–725. doi:10.1145/3470496.3527415
- [7] Florian Krieger, Florian Hirmer, Ahmet Can Mert, and Sujoy Sinha Roy. 2024. Aloha-HE: A Low-Area Hardware Accelerator for Client-Side Operations in Homomorphic Encryption. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6. doi:10.23919/DATE58400.2024.10546608
- [8] Jaehyeok Lee, Phap Ngoc Duong, and Hanho Lee. 2023. Configurable Encryption and Decryption Architectures for CKKS-Based Homomorphic Encryption. *Sensors* 23, 17 (2023), 7389. doi:10.3390/s23177389
- [9] Ahmet Can Mert, Erdinç Öztürk, and Erkey Savaş. 2020. Design and Implementation of Encryption/Decryption Architectures for BFV Homomorphic Encryption Scheme. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 2 (2020), 353–362. doi:10.1109/TVLSI.2019.2943127
- [10] Zhenyuan Ruan, Tong He, and Jason Cong. 2019. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 379–394. <https://www.usenix.org/conference/atc19/presentation/ruan>
- [11] SNIA. 2025. *Computational Storage Architecture and Programming Model* (version 1.1 ed.). SNIA. <https://www.snia.org/sites/default/files/technical-work/computational/release/SNIA-Computational-Storage-Architecture-and-Programming-Model-1.1.pdf> Released May 31, 2025.
- [12] Infall Syafalni, Najmi Azzahra Feryputri, Nana Sutisna, Nur Ahmadi, Rahmat Mulyawan, Tutun Juhana, and Trio Adiono. 2026. Multicore Encryption-Decryption Hardware Accelerator for Torus-Based RLWE FHE. In *2026 IEEE Symposium on Low-Power and High-Speed Chips and Systems (COOL CHIPS)*. IEEE, 1–3. doi:10.1109/COOLCHIPS68842.2026.11556998
- [13] Michiel Van Beirendonck, Jan-Pieter D'Anvers, Furkan Turan, and Ingrid Verbauwhede. 2023. FPT: A Fixed-Point Accelerator for Torus Fully Homomorphic Encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 741–755. doi:10.1145/3576915.3623159
- [14] McKenzie van der Hagen and Brandon Lucia. 2022. Client-Optimized Algorithms and Acceleration for Encrypted Compute Offloading. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 683–696. doi:10.1145/3503222.3507737
- [15] Linus Y. Wong, Jiali Zhang, and Jing (Jane) Li. 2024. DONGLE 2.0: Direct FPGA-Orchestrated NVMe Storage for HLS. *ACM Transactions on Reconfigurable Technology and Systems* 17, 3, Article 45 (2024), 32 pages. doi:10.1145/3650038
- [16] Zhihan Xu, Rajgopal Kannan, and Viktor Prasanna. 2026. HERA: A Bandwidth-efficient Accelerator for Fully Homomorphic Encryption on HBM-enabled FPGA. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '26)*. Association for Computing Machinery, New York, NY, USA, 265–276. doi:10.1145/3748173.3779201
- [17] Sungwoong Yune, Hyejeong Lee, Adiwena Putra, Hyunjun Cho, Cuong Duong Manh, Jaeho Jeon, and Joo-Young Kim. 2025. ABC-FHE: A Resource-Efficient Accelerator Enabling Bootstrappable Parameters for Client-Side Fully Homomorphic Encryption. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7. doi:10.1109/DAC63849.2025.11132592
- [18] Minxuan Zhou, Yujin Nam, Pranav Gangwar, Weihong Xu, Arpan Dutta, Kartikeyan Subramanyam, Chris Wilkerson, Rosario Cammarota, Saransh Gupta, and Tajana Rosing. 2023. FHEmem: A Processing In-Memory Accelerator for Fully Homomorphic Encryption. arXiv preprint arXiv:2311.16293. <https://arxiv.org/abs/2311.16293v1>

A Reproducibility Details

Writing guide: Provide commands, TFHE parameters, input data, preprocessing settings, FPGA build details, and timing procedures. Document the HPU operation, ciphertext exchange, and verification of decrypted and persisted results.